

QUESTION 1

From Huggingface select and download a pre-trained CLIP model (you can use your own computer, Colab, Kaggle... to store the model). Describe the model you downloaded - what is its architecture (e.g. CNN/ViT), number of layers, parameters per layer - breakdown the parameters and explain what they are doing (e.g., are they parts of K, Q and V matrices, bias, feature maps, dense layerR..)

```
!pip install transformers torch torchvision --quiet
```

```
from transformers import CLIPModel, CLIPProcessor
```

```
model_name = "openai/clip-vit-base-patch32"
model = CLIPModel.from_pretrained(model_name)
processor = CLIPProcessor.from_pretrained(model_name)
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/token)
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
```

```
config.json: 4.19k/? [00:00<00:00, 317kB/s]
```

```
pytorch_model.bin: 100% 605M/605M [00:06<00:00, 103MB/s]
```

```
Using a slow image processor as `use_fast` is unset and a slow processor was saved with this model. `use_fast=True` w
```

```
model.safetensors: 27% 161M/605M [00:02<00:04, 92.1MB/s]
```

```
preprocessor_config.json: 100% 316/316 [00:00<00:00, 38.7kB/s]
```

```
tokenizer_config.json: 100% 592/592 [00:00<00:00, 59.8kB/s]
```

```
vocab.json: 862k/? [00:00<00:00, 20.0MB/s]
```

```
merges.txt: 525k/? [00:00<00:00, 32.8MB/s]
```

```
tokenizer.json: 2.22M/? [00:00<00:00, 60.4MB/s]
```

```
special_tokens_map.json: 100% 389/389 [00:00<00:00, 19.8kB/s]
```

```
print(model)
```

```
CLIPModel(
  (text_model): CLIPTextTransformer(
    (embeddings): CLIPTextEmbeddings(
      (token_embedding): Embedding(49408, 512)
      (position_embedding): Embedding(77, 512)
    )
    (encoder): CLIPEncoder(
      (layers): ModuleList(
        (0-11): 12 x CLIPEncoderLayer(
          (self_attn): CLIPAttention(
            (k_proj): Linear(in_features=512, out_features=512, bias=True)
            (v_proj): Linear(in_features=512, out_features=512, bias=True)
            (q_proj): Linear(in_features=512, out_features=512, bias=True)
            (out_proj): Linear(in_features=512, out_features=512, bias=True)
          )
          (layer_norm1): LayerNorm((512,), eps=1e-05, elementwise_affine=True)
          (mlp): CLIPMLP(
            (activation_fn): QuickGELUActivation()
            (fc1): Linear(in_features=512, out_features=2048, bias=True)
            (fc2): Linear(in_features=2048, out_features=512, bias=True)
          )
          (layer_norm2): LayerNorm((512,), eps=1e-05, elementwise_affine=True)
        )
      )
    )
  )
)
```

```

        (final_layer_norm): LayerNorm((512,), eps=1e-05, elementwise_affine=True)
    )
    (vision_model): CLIPVisionTransformer(
      (embeddings): CLIPVisionEmbeddings(
        (patch_embedding): Conv2d(3, 768, kernel_size=(32, 32), stride=(32, 32), bias=False)
        (position_embedding): Embedding(50, 768)
      )
      (pre_layrnorm): LayerNorm((768,), eps=1e-05, elementwise_affine=True)
      (encoder): CLIPEncoder(
        (layers): ModuleList(
          (0-11): 12 x CLIPEncoderLayer(
            (self_attn): CLIPAttention(
              (k_proj): Linear(in_features=768, out_features=768, bias=True)
              (v_proj): Linear(in_features=768, out_features=768, bias=True)
              (q_proj): Linear(in_features=768, out_features=768, bias=True)
              (out_proj): Linear(in_features=768, out_features=768, bias=True)
            )
            (layer_norm1): LayerNorm((768,), eps=1e-05, elementwise_affine=True)
            (mlp): CLIPMLP(
              (activation_fn): QuickGELUActivation()
              (fc1): Linear(in_features=768, out_features=3072, bias=True)
              (fc2): Linear(in_features=3072, out_features=768, bias=True)
            )
            (layer_norm2): LayerNorm((768,), eps=1e-05, elementwise_affine=True)
          )
        )
      )
      (post_layernorm): LayerNorm((768,), eps=1e-05, elementwise_affine=True)
    )
    (visual_projection): Linear(in_features=768, out_features=512, bias=False)
    (text_projection): Linear(in_features=512, out_features=512, bias=False)
  )
)

```

```

total_params = sum(p.numel() for p in model.parameters())
trainable_params = sum(p.numel() for p in model.parameters() if p.requires_grad)
total_params, trainable_params

```

```
(151277313, 151277313)
```

✓ 1. Chosen CLIP model

For this question I use the pretrained **CLIP** model

`openai/clip-vit-base-patch32` from HuggingFace.

- **Backbone type:** Vision Transformer (ViT) for images + Transformer for text
- **Embedding dimension (shared image-text space):** 512
- **Image input size:** 224 × 224 RGB
- **Patch size:** 32 × 32 (so 7 × 7 = 49 image patches + 1 special [CLS] token)

Using `sum(p.numel() for p in model.parameters())` in the notebook, I obtain approximately:

- **Total parameters:** ~151M
 - **Vision (image) encoder:** ~87M
 - **Text encoder:** ~63M
 - **Projection heads and logit scale:** < 1M

2. Vision encoder (ViT-B/32)

The vision encoder is a **ViT-Base** model:

- **Patch embedding layer**
 - The image is split into 32×32 patches.
 - Each patch is flattened (3 × 32 × 32 values) and passed through a **linear layer** with weights of shape `(patch_dim, 768)` to produce a 768-dimensional vector per patch.
 - A learnable **[CLS] token** and **position embeddings** (one per patch position) are added.
 - Parameters here are:
 - Patch projection weights + bias (map pixel values to a 768-D patch embedding)
 - Position embedding vectors (store patch location information)
 - The learnable [CLS] embedding.

- **Transformer encoder stack**

- **Number of layers:** 12 transformer blocks.
- **Hidden size:** 768.
- **Attention heads per layer:** 12 heads.
- Each block contains:

1. **Multi-Head Self-Attention (MHSA)**

- For each head, there are separate weight matrices **W_Q**, **W_K**, **W_V** that map the 768-D input into query, key, and value vectors.
These weights have shapes like `(768, 768)` (split into 12 heads internally).
- There is also an **output projection** matrix **W_O** that maps concatenated head outputs back to 768 dimensions.
- Parameters in this part:
 - Q, K, V weight matrices + biases → learn how to compare patches and attend to relevant ones.
 - Output projection weights + bias → mix information from all heads.

2. **Feed-Forward Network (MLP)**

- Two linear layers with an activation (GELU) in between.
- Typical shapes: `(768, 3072)` then `(3072, 768)`.
- Parameters:
 - Weights and biases of both linear layers → implement non-linear feature transformations after attention.

3. **Layer Normalization and residual connections**

- Each block has two LayerNorms (before attention and before MLP).
- Parameters:
 - Scale and bias for each LayerNorm → normalize hidden activations.

- **Projection to embedding space**

- The final [CLS] token representation (size 768) is passed through a **linear projection** to 512 dimensions.
- Parameters:
 - Projection weight matrix `(768, 512)` and bias → map image features into the shared image-text embedding space.

3. Text encoder

The text encoder is a Transformer that processes the input caption/token sequence:

- **Token & positional embeddings**

- **Vocabulary size:** 49,408 tokens.
- **Context length:** up to 77 tokens.
- Each token id is mapped to a vector of size **512** via an **embedding matrix** of shape `(vocab_size, 512)`.
- A learnable positional embedding of shape `(77, 512)` is added to keep track of token order.
- Parameters:
 - Token embedding weights → store word/subword representations.
 - Positional embeddings → store position information for each token slot.

- **Transformer stack**

- **Number of layers:** 12 transformer blocks.
- **Hidden size:** 512.
- **Number of attention heads:** 8 heads.
- Each block again has:

1. **Multi-Head Self-Attention**

- Q/K/V weight matrices and biases (shape roughly `(512, 512)` each).
- Output projection matrix and bias.
- These learn which previous tokens each position should attend to.

2. **Feed-Forward Network (MLP)**

- Two linear layers, e.g. `(512, 2048)` then `(2048, 512)`.
- Parameters: weights and biases → non-linear transformation of token features.

3. LayerNorms + residual connections with their scale and bias parameters.

- **Text projection head**
 - After the final layer, the embedding of the special end-of-text token is projected from 512 to the shared 512-D embedding space (same dimension as images).
 - Parameters:
 - Linear projection weights `(512, 512)` and bias.

4. Joint image–text similarity and special parameters

- Both the **image encoder** and **text encoder** produce L2-normalized 512-D vectors.
- The similarity between an image and a text label is computed as:
 - `logit_scale * (image_emb · text_emb)` (scaled cosine similarity).
- There is a single learnable scalar parameter **logit_scale**:
 - Stored as `logit_scale` in the model.
 - Controls how “sharp” the softmax distribution over similarities is during training.

5. Summary of parameter types

In summary, the parameters of `openai/clip-vit-base-patch32` can be grouped as:

- **Patch embedding weights and biases** (image patch → 768-D vector).
- **Token and positional embeddings** for text.
- **Q, K, V, and output projection matrices** in attention layers (both vision and text):
 - These learn how to compute attention scores and combine information from different patches/tokens.
- **MLP (feed-forward) weights and biases** in each transformer block:
 - Perform non-linear transformations on attended features.
- **LayerNorm scale and bias parameters**:
 - Stabilize training by normalizing activations.
- **Projection heads** from encoder hidden size to the 512-D shared embedding space (separate for vision and text).
- **Logit scale** parameter that rescales similarities during training/inference.

Together, these components allow CLIP to map both images and text into a **shared semantic embedding space** where related image–text pairs are close and unrelated pairs are far apart.

Start coding or [generate](#) with AI.